# Artificial Intelligence
# Introduction



**Prof. Dr. Fazlul Hasan Siddiqui**
Dept. of CSE, DUET, Gazipur
BSc:IUT; MSc:BUET; PhD:ANU (Australia)
siddiqui@duet.ac.bd
Credit: Prof. Joerge Hoffman & Prof. Malte Helmert

# Textbook – The Famous AIMA Book

## (Fourth edition, 2020)

## by Stuart Russell and Peter Norvig

The leading textbook in Artificial Intelligence, used in over **1400** schools in over **120** countries.

## Table of Contents


Preface (pdf); Contents with subsections

**I Artificial Intelligence**
1 Introduction ... 1
2 Intelligent Agents ... 36

**II Problem-solving**
3 Solving Problems by Searching ... 63
4 Search in Complex Environments ... 110
5 Adversarial Search and Games ... 146
6 Constraint Satisfaction Problems ... 180

**III Knowledge, reasoning, and planning**
7 Logical Agents ... 208
8 First-Order Logic ... 251
9 Inference in First-Order Logic ... 280
10 Knowledge Representation ... 314
11 Automated Planning ... 344

**IV Uncertain knowledge and reasoning**
12 Quantifying Uncertainty ... 385
13 Probabilistic Reasoning ... 412
14 Probabilistic Reasoning over Time ... 461
15 Probabilistic Programming ... 500
16 Making Simple Decisions ... 528
17 Making Complex Decisions ... 562
18 Multiagent Decision Making ... 599

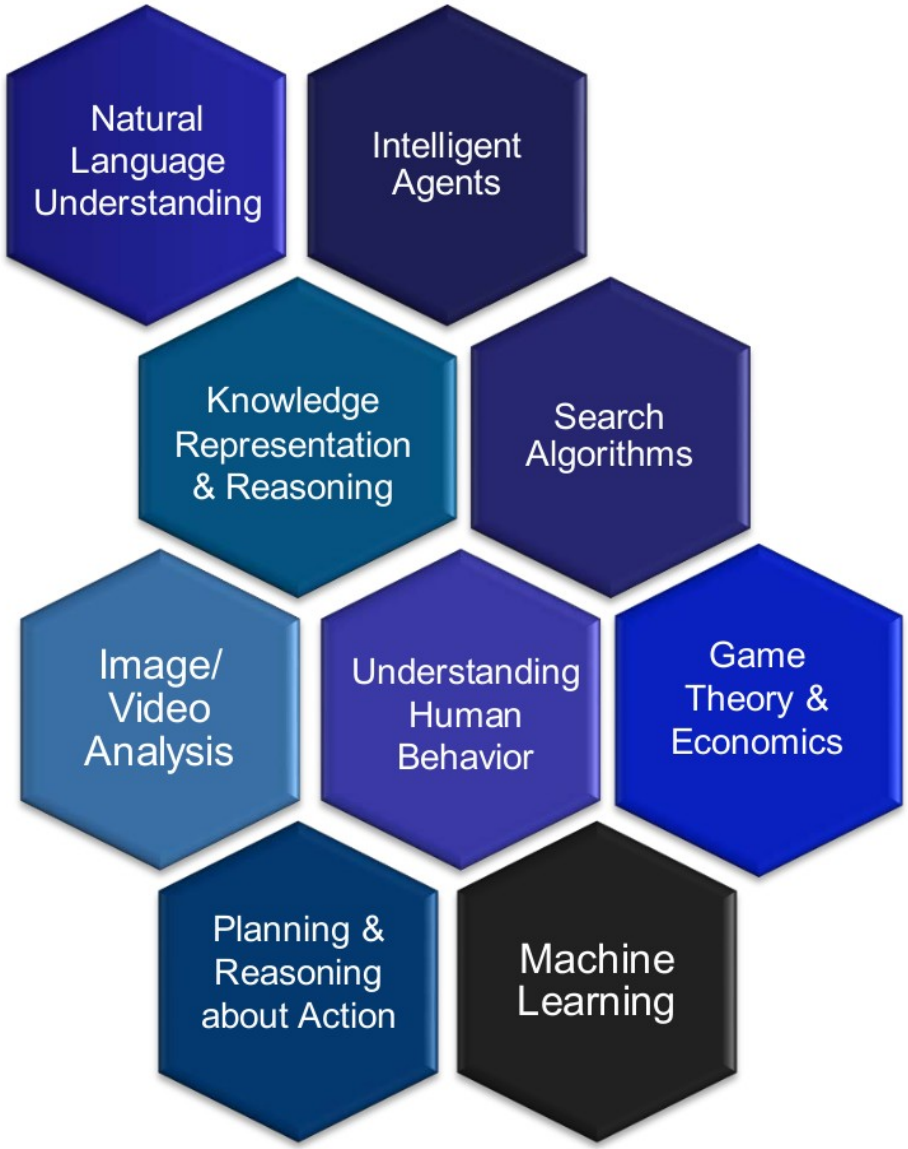
**V Machine Learning**
19 Learning from Examples ... 651
20 Learning Probabilistic Models ... 721
21 Deep Learning ... 750
22 Reinforcement Learning ... 789

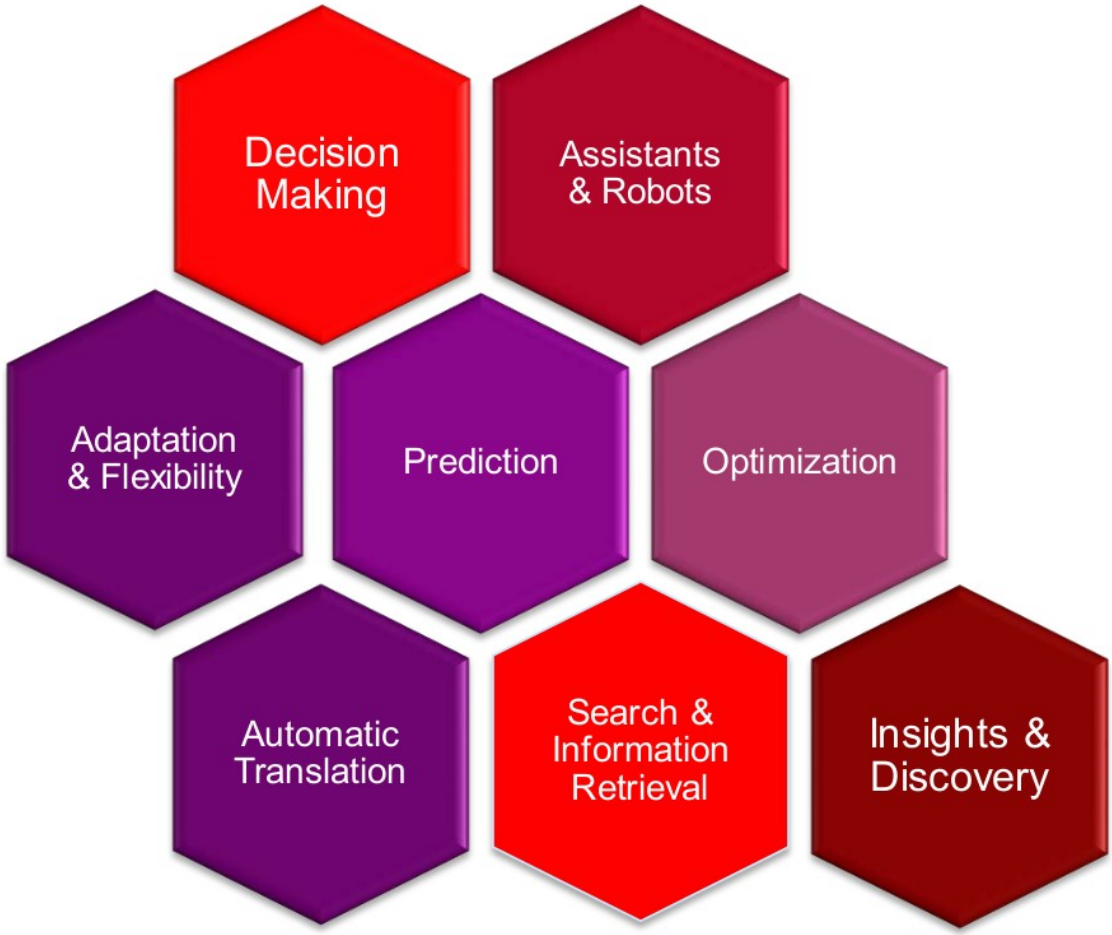
**VI Communicating, perceiving, and acting**
23 Natural Language Processing ... 823
24 Deep Learning for Natural Language Processing ... 856
25 Computer Vision ... 881
26 Robotics ... 925

**VII Conclusions**
27 Philosophy, Ethics, and Safety of AI ... 981
28 The Future of AI ... 1012
Appendix A: Mathematical Background ... 1023
Appendix B: Notes on Languages and Algorithms ... 1030
Bibliography ... 1033 (pdf and bib data)
Index ... 1069 (pdf)

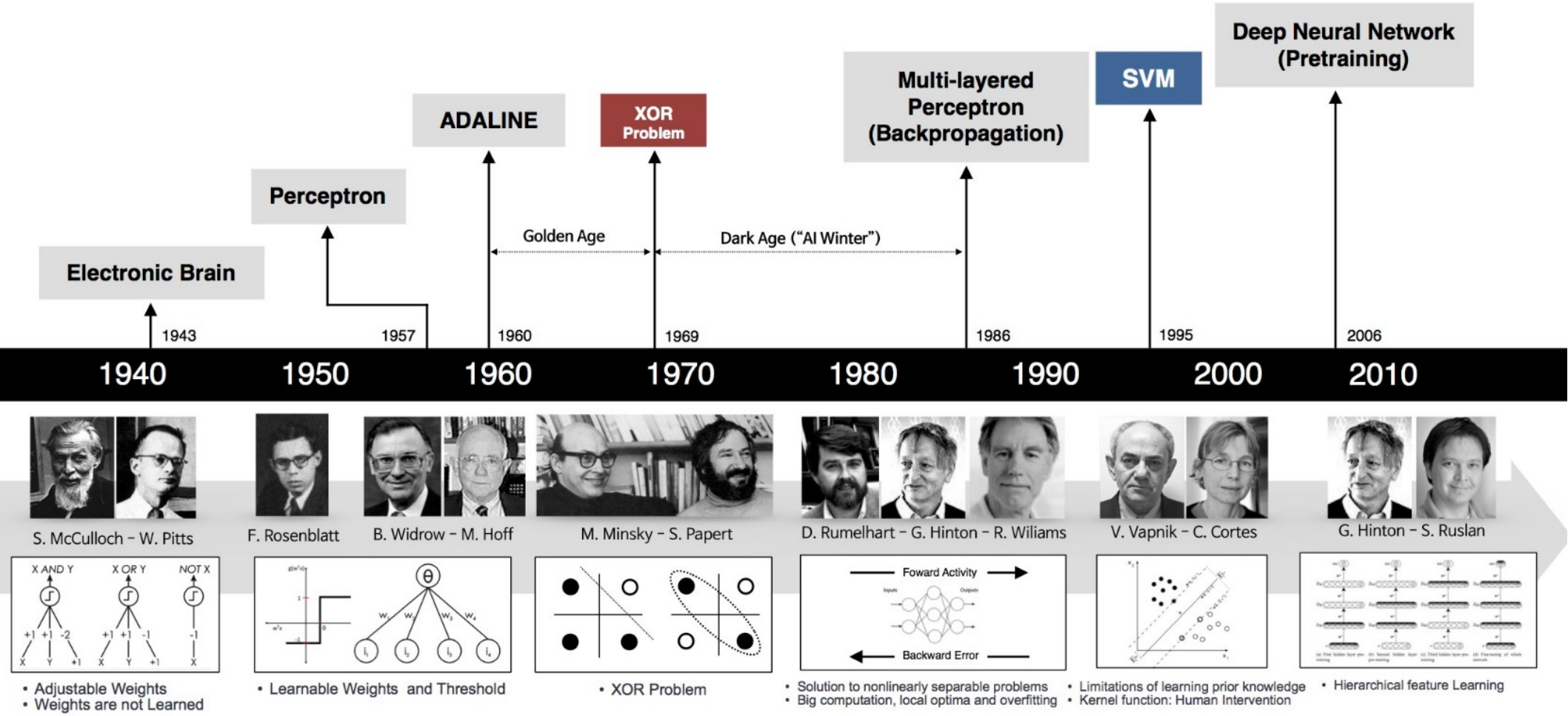
Exercises (website)
Figures (pdf)
Code (website); Pseudocode (pdf)

# Major Technologies & Application Areas



# Dramatic Progress over the Last Decade – NN




*https://beamandrew.github.io/deeplearning/2017/02/23/deep_learning_101_part1.html*

# Dramatic Progress over the Last Decade - Search

Systematic Search

Alpha Beta Pruning

$10^5/10^3$

1940s-1970s

Temporal Difference Learning

$10^{144}$

1992

TD-Gammon



IBM

Heuristic Search and Knowledge

$10^{123}$

1996/97

Deep Blue



IBM

Conflict Learning

Monte-Carlo Tree Search

2012

Watson



IBM Watson

Reinforcement Learning

Deep Learning

$10^{360}$

2016

AlphaGo



> 20 People

Google

Discounted Regret Minimization

Imperfect Information Games

2017

Libratus



Carnegie Mellon University
Computer Science Department

Population-based/ Multi-Agent RL

Long-/Shortterm Planning

Policy Distillation

2019

AlphaStar



> 40 People

DeepMind

TicTacToe/Checkers



Backgammon



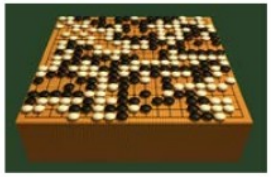
Chess



Jeopardy



Go



Poker



Starcraft II



© Jana Koehler

# What is AI !!
# Humanly vs. Rationally || Thinking vs. Acting

| | |
|---|---|
| **Thinking Humanly**<br><br>"The exciting new effort to make computers think . . . machines with minds, in the full and literal sense." (Haugeland, 1985)<br><br>"[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . ." (Bellman, 1978) | **Thinking Rationally**<br><br>"The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985)<br><br>"The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992) |
| **Acting Humanly**<br><br>"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990)<br><br>"The study of how to make computers do things at which, at the moment, people are better." (Rich and Knight, 1991) | **Acting Rationally**<br><br>"Computational Intelligence is the study of the design of intelligent agents." (Poole et al., 1998)<br><br>"AI . . . is concerned with intelligent behavior in artifacts." (Nilsson, 1998) |

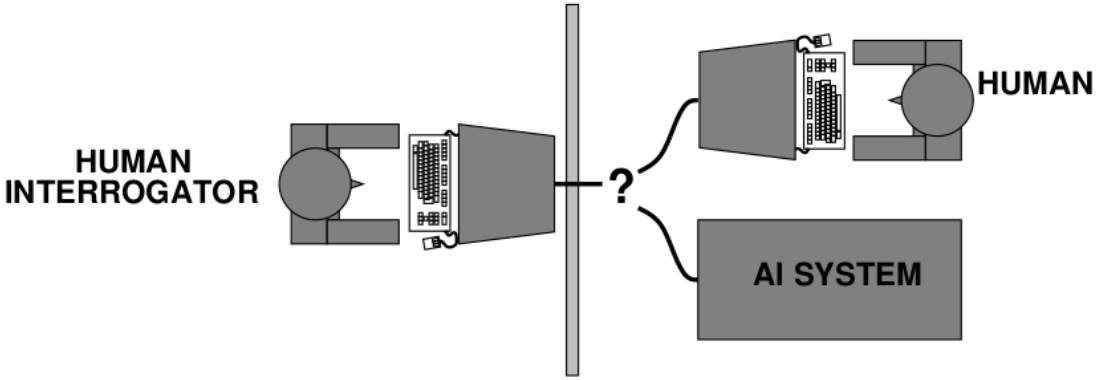
# Thinking Humanly

- This requires "getting inside" of the human mind to see how it works and then comparing our computer programs to this,
  - like what cognitive science attempts to do.
- Another way to do this is to observe a human problem solving and argue that one's programs go about problem solving in a similar way.
- Not important: Being able to solve problems correctly.
- Important: Being able to solve problems like a human would.
- Example: GPS (General Problem Solver) was an early computer program that attempted to model human thinking. The developers were not so much interested in whether or not GPS solved problems correctly. They were more interested in showing that it solved problems like people, going through the same steps and taking around the same amount of time to perform those steps.
- This is what cognitive science and cognitive psychology attempt to do.
  - Introspection (observing ourselves); Psychological experiments; Brain research.

# Acting Humanly

Alan Turing, Computing Machinery and Intelligence (1950):

- from "can machines think?"
  to "can machines act intelligently?"
- operationalization: the imitation game







//www.imdb.com/title/tt2084970/

**To pass this test requires:**

- natural language processing
- knowledge representation
- automated reasoning
- machine learning

**A total Turing test would also require:**

- computer vision
- robotics




http://www.imdb.com/title/tt0470752/

# Thinking Rationally

- Aristotle was one of the first to attempt to codify "thinking".
  - His syllogisms provided patterns of argument structure that always gave correct conclusions, giving correct premises.
- Example: All computers use energy. Using energy always generates heat. Therefore, all computers generate heat.
- This initiate the field of logic.
  - By 1965, programs existed that could, given enough time and memory, take a description of the problem in logical notation and find the solution, if one existed.
- There are two main obstacles to this approach:
  - it is difficult to make informal/commonsense knowledge precise enough to use the logicist approach particularly when there is uncertainty in the knowledge.
  - there is a big difference between being able to solve a problem in principle and doing so in practice.

# Acting Rationally

- Acting rationally means acting so as to achieve one's goals, given one's beliefs.
- Rational thinking is a prerequisite for rational acting, although it is not a necessary condition
  - what to do, for example, when we must make a decision faced with insufficient information? e.g., reflex actions.
- The study of AI as rational agent design has two advantages:
  - It is more general than the logical approach because correct inference is only a useful mechanism for achieving rationality, not a necessary one.
  - It is more amenable to scientific development than approaches based on human behaviour or human thought because a standard of rationality can be defined independent of humans.
- Achieving perfect rationality in complex environments is not possible because the computational demands are too high. However, we will study perfect rationality as a starting place.

# Nature of Intelligence

AI researchers study the **nature of intelligence**. They don't (necessarily) try to build androids because

1. It was the study of the principles of aerodynamics, not the attempt to make mechanical birds, that enabled human flight.
2. We already know how to make new humans anyway (sorry, no hyperlinks here).

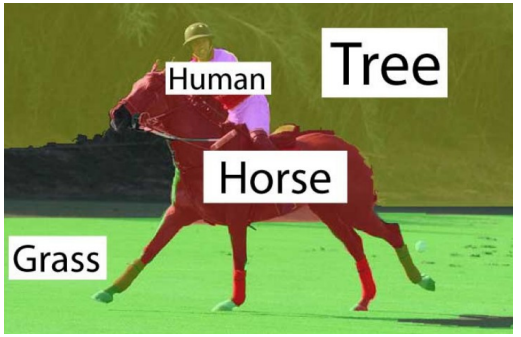
Intelligence involves sensing, thinking, and acting.

| SENSING | THINKING | ACTING |
|---|---|---|
| Translation of sensory inputs (percepts) into a conceptual representation<br>• Computer Vision<br>• Speech Recognition<br>• Language Understanding | Manipulation of the conceptual representation<br>• Knowledge Representation<br>• Problem Solving/Planning<br>• Learning (making improvements based on the results of past actions) | Translation of intent into (physical) actions (reflexive or deliberative)<br>• Robotics<br>• Speech and Language Synthesis |

# Machine vs Human

- Machines have far superior computational abilities than humans
  - Square root of 964,324 ?
- Machines can sort through enormous amounts of data.
  - While a doctor makes a diagnosis in ~10 minutes, AI system makes a million.
- Humans, in general, are good at ?

## Scene Understanding




## Taxi Driving



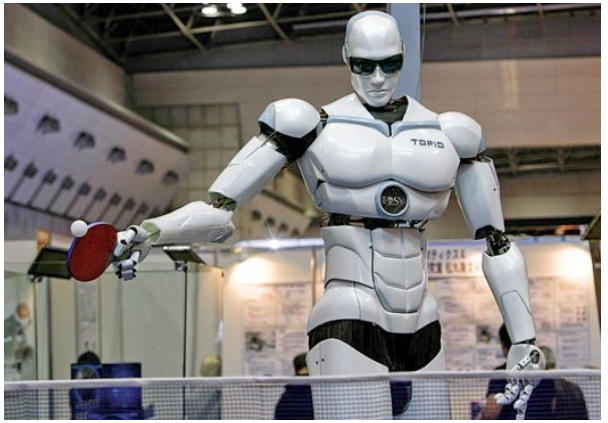

## Dialogue Processing



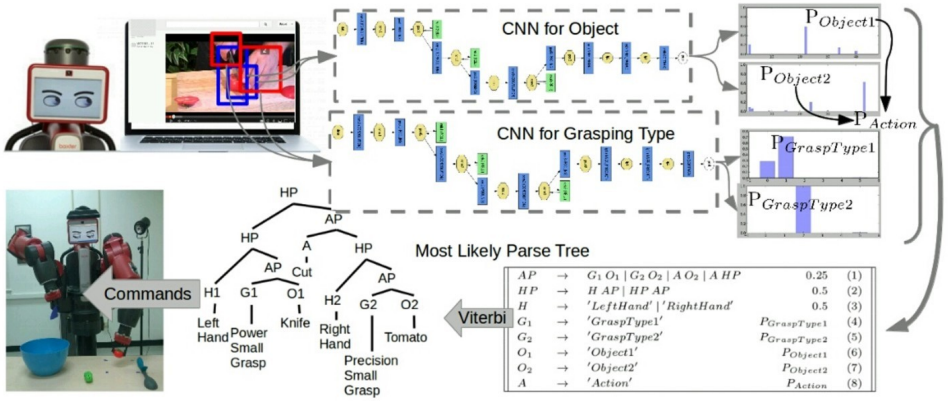
## Identify the digits



## Ping-Pong playing robot



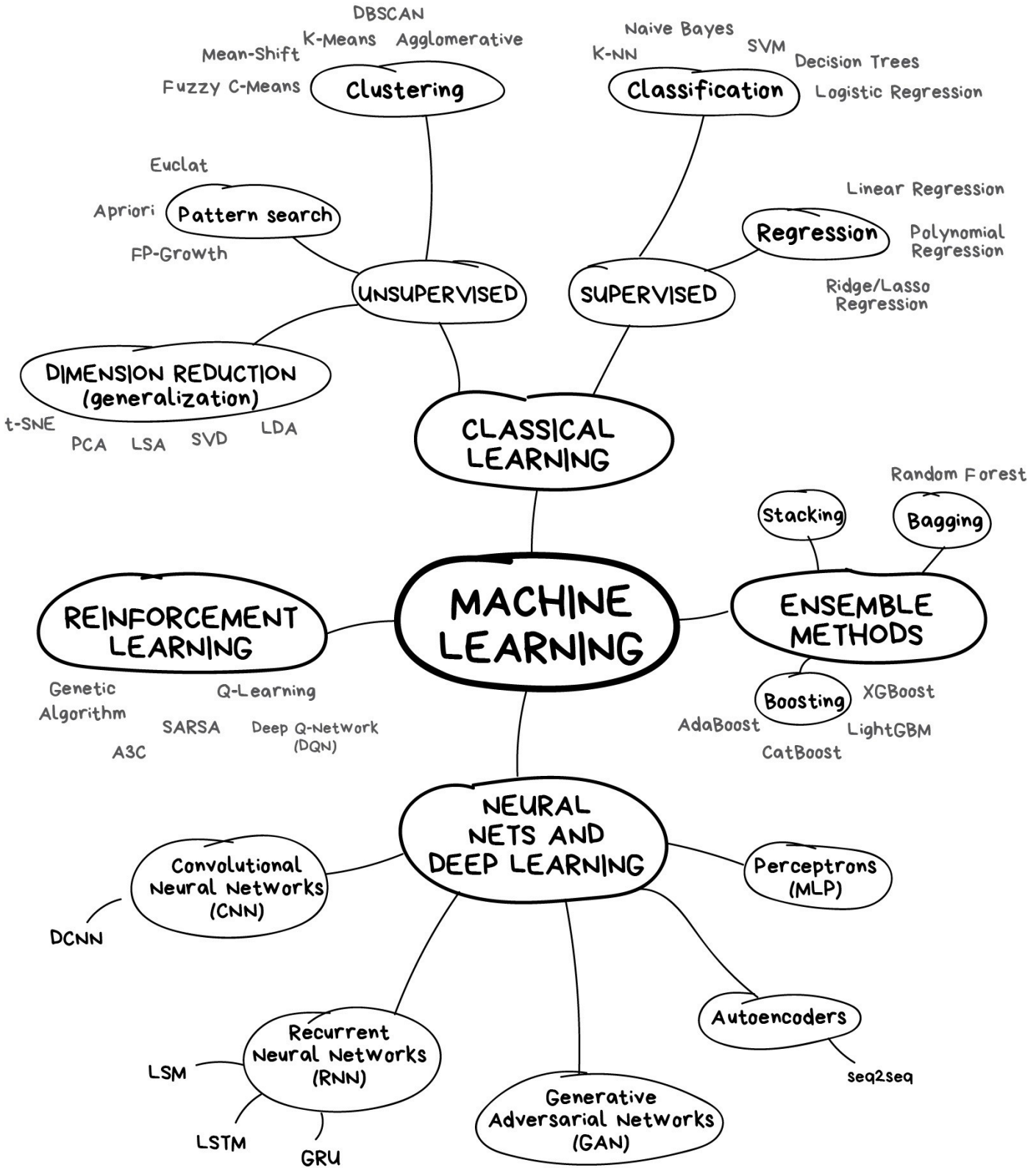
## Learn Cook from youtube




## Perform clustering

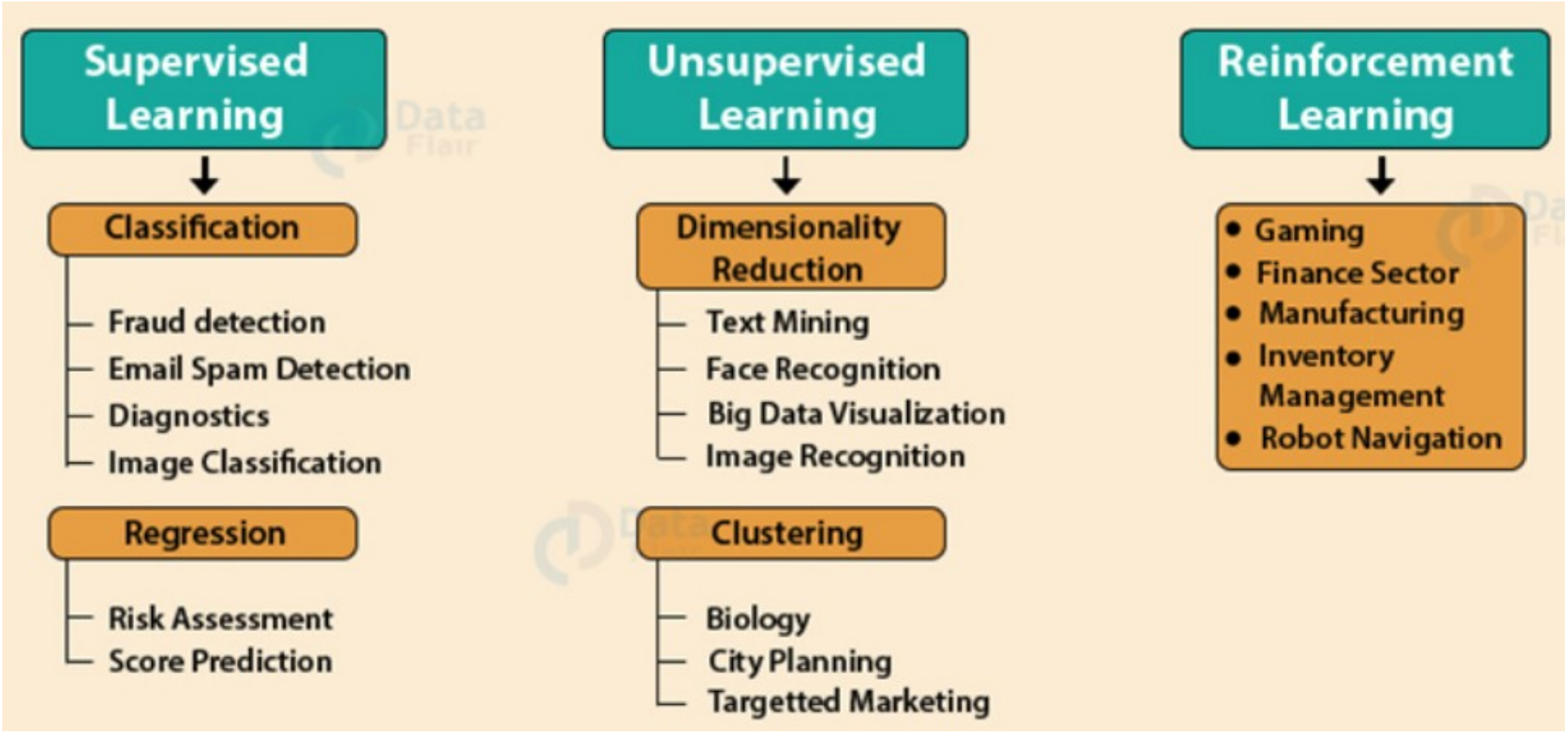


# Machine Learning Algorithms

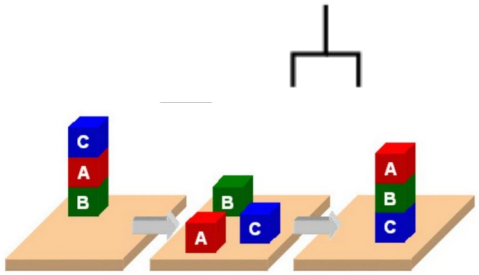


# Machine Learning Applications

Why not allow machines to do the heavy thinking for us in order to make better decisions that, in general, people do better ?



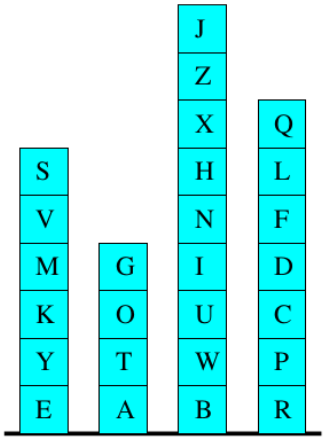
# Problems Deep Learning cannot Solve



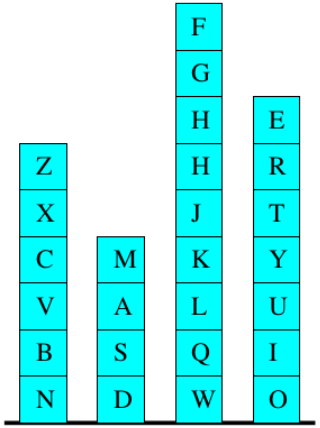

- $n$ blocks, 1 hand.
- A single action either takes a block with the hand or puts a block we're holding onto some other block/the table.

Find the shortest sequence of actions that transforms an **arbitrary** initial configuration of blocks into another **arbitrary** goal configuration

| blocks | states | blocks | states |
|---|---|---|---|
| 1 | 1 | 9 | 4596553 |
| 2 | 3 | 10 | 58941091 |
| 3 | 13 | 11 | 824073141 |
| 4 | 73 | 12 | 12470162233 |
| 5 | 501 | 13 | 202976401213 |
| 6 | 4051 | 14 | 3535017524403 |
| 7 | 37633 | 15 | 65573803186921 |
| 8 | 394353 | 16 | 1290434218669921 |



Initial State



Goal State

→ State spaces typically are huge even for simple problems.

→ In other words: Even solving "simple problems" automatically (without help from a human) requires a form of intelligence. With blind search, even the largest super-computer in the world won't scale beyond 20 blocks!

# Planning & Scheduling: Field of AI

## Example (Agricultural experiment design)

| | plot1 | plot2 | plot3 | plot4 | plot5 | plot6 | plot7 |
|---|---|---|---|---|---|---|---|
| barley | ✓ | ✓ | ✓ | – | – | – | – |
| corn | ✓ | – | – | ✓ | ✓ | – | – |
| millet | ✓ | – | – | – | – | ✓ | ✓ |
| oats | – | ✓ | – | ✓ | – | ✓ | – |
| rye | – | ✓ | – | – | ✓ | – | ✓ |
| spelt | – | – | ✓ | ✓ | – | – | ✓ |
| wheat | – | – | ✓ | – | ✓ | ✓ | – |

**Constraints** to be **satisfied**:

1. Equal growth load: Every plot grows 3 grains.
2. Equal sample size: Every grain is grown in 3 plots.
3. Balance: Every grain pair is grown in 1 common plot.

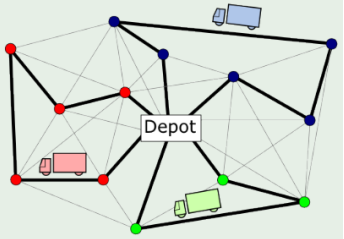
**Instance**: 7 plots, 7 grains, 3 grains/plot, 3 plots/grain, balance 1.

## Example (Vehicle routing: parcel delivery)

**Given** a depot with parcels for clients and a vehicle fleet, **find** which vehicle visits which client when.

**Constraints** to be **satisfied**:

1. All parcels are delivered on time.
2. No vehicle is overloaded.
3. Driver regulations are respected.
4. ...




**Objective function** to be **minimised**:

- Cost: the total fuel consumption and driver salary.

## Example (Doctor rostering)

| | Mon | Tue | Wed | Thu | Fri | Sat | Sun |
|---|---|---|---|---|---|---|---|
| Doctor A | call | none | oper | none | oper | none | none |
| Doctor B | app | call | none | oper | none | none | call |
| Doctor C | oper | none | call | app | app | call | none |
| Doctor D | app | oper | none | call | oper | none | none |
| Doctor E | oper | none | oper | none | call | none | none |

**Constraints** to be **satisfied**:

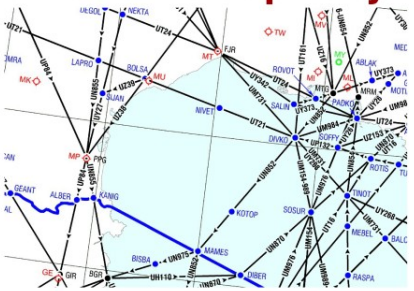
1. #doctors-on-call / day $= 1$
2. #operations / workday $\leq 2$
3. #operations / week $\geq 7$
4. #appointments / week $\geq 4$
5. day off after operation day
6. ...



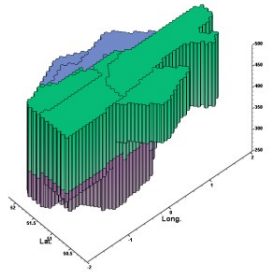
**Objective function** to be **minimised**:

- Cost: ...

**Demand vs capacity**



**Airspace sectorisation**

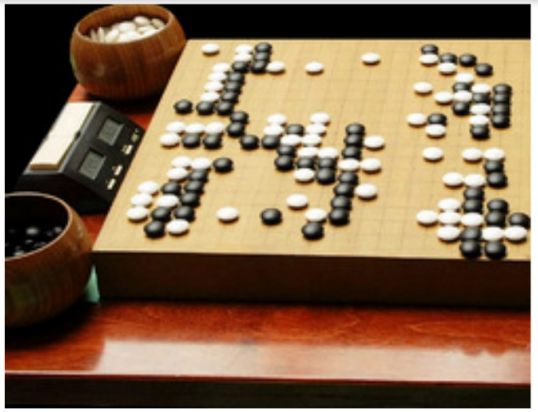


# Planning & Scheduling: Field of AI

## Games




IBM's Deep Blue wins 6 game match against Garry Kasparov ('97)





- Go: 2008: MoGo beats Pro (8P) in 9-stone game
- Poker: Next big frontier for AI in games



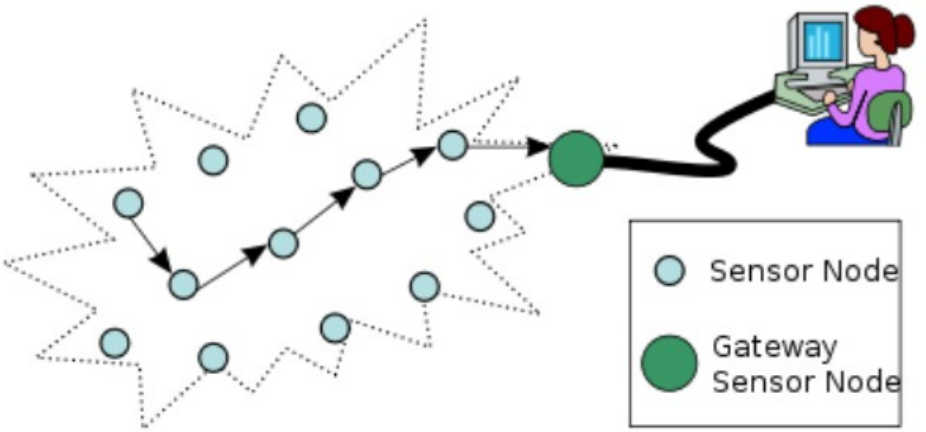


# Planning & Scheduling: Field of AI

## Robot task sequencing



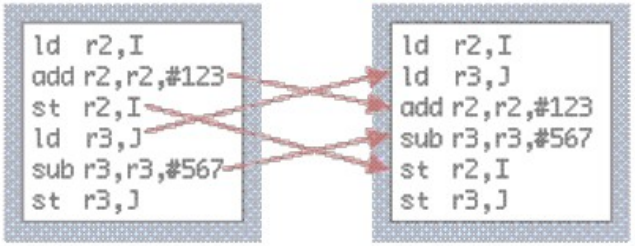
## Sensor net configuration



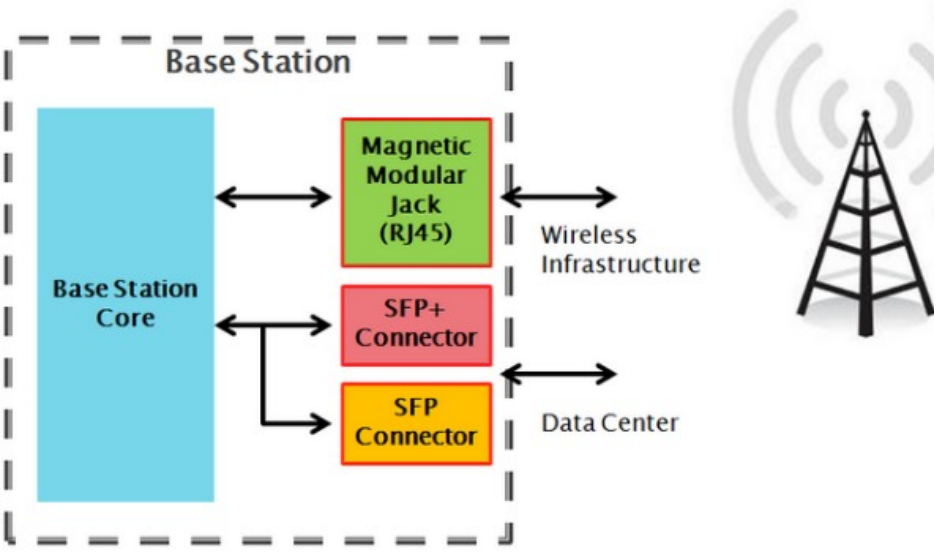

## Compiler design

COMPILERS
FOR INSTRUCTION SCHEDULING

C Compiler
C++ Compiler




## Base station testing



# Planning & Scheduling: Field of AI

## School timetabling

| | Monday | Tuesday | Wednesday | Thursday | Friday |
|---|---|---|---|---|---|
| 9.00 | MT2202 Ordinary Differential Equations FTh1 | | LABCS2072 Computer Graphics (G) Dual | MT2282 Numerical Analysis F Williamson, G03 | |
| 10.00 | XMT2202 Ordinary Differential Equations NG15 / Roscoe, 2.3 | | LABCS2072 Computer Graphics (G) Dual | XMT2202 Ordinary Differential Equations Simon Engineering, Basement Theatre 3A<br>XMT2282 Numerical Analysis F LG20 | XMT2202 Ordinary Differential Equations NG15 |
| 11.00 | CS2012 Algorithms and Data Structures 1.1 | | XMT2212 Further Linear Algebra 1.5 | | MT2202 Ordinary Differential Equations Stopford, Theatre 1 |
| 12.00 | MT2212 Further Linear Algebra Roscoe, Theatre A | MT2282 Numerical Analysis F Williamson, G03 | CS2072 Computer Graphics 1.1 | | MT2212 Further Linear Algebra Stopford, Theatre 1 |
| 1.00 | | | PASS Peer Assisted Study 1607 / LF15 / LF17 / 1606 | | XMT2212 Further Linear Algebra Simon Engineering, Basement Theatre 4A |
| 2.00 | CS2072 Computer Graphics 1.1 | | | XMT2212 Further Linear Algebra NG17 | |
| 3.00 | | CSTUT Tutorial | | | |
| 4.00 | | CS2012 Algorithms and Data Structures 1.1 | | | |

## Sports tournament design



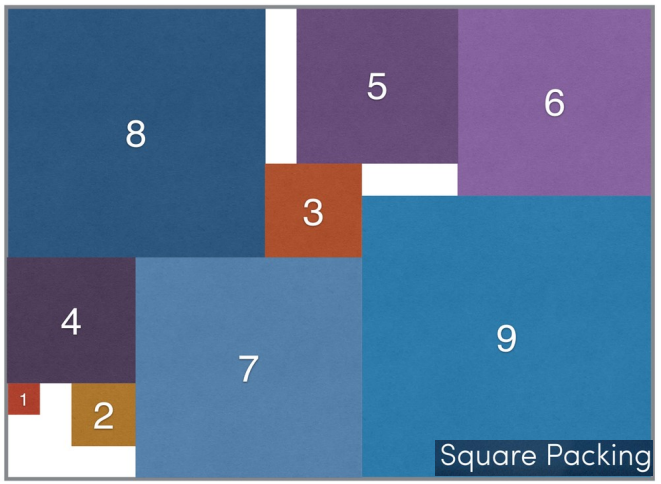

## Security: SQL injection?



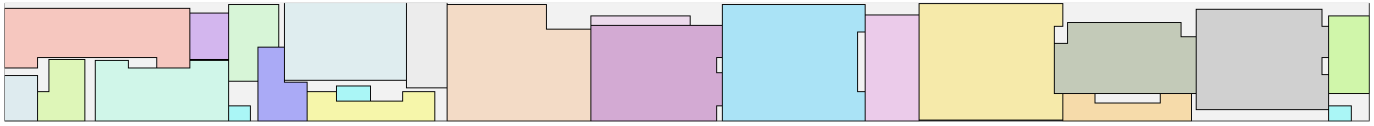

## Container packing



# Planning & Scheduling: Field of AI




- Complex packing problem
- Room shapes are complex
- Stair carpets have many cutting options
- Pattern of carpet is important





| Task | Description | Duration | Predecessor |
|---|---|---|---|
| a | Erecting Walls | 7 | none |
| b | Carpentry for Roof | 3 | a |
| c | Roof | 1 | b |
| d | Installations | 8 | a |
| e | Facade Painting | 2 | c, d |
| f | Windows | 1 | c, d |
| g | Garden | 1 | c, d |
| h | Ceilings | 3 | a |
| i | Painting | 2 | f, h |
| j | Moving in | 1 | i |

# Planning & Scheduling: Field of AI

What have all this in common ?

- Large search space
- well-identified “goal”
  - Notion of solution is easy to define (declaratively)
- But we don’t know how to reach it
- No algorithm to build a solution incrementally
- Hence:
  - need to explore the search space
  - Either exhaustively or in an “intelligent”, “guided” manner

# Solving Real-World Problems

**Modeling**
(Statistics / D.Tree / D.Graph / Logics / Constraint/ PDDL)

**+**

**Solving**
(Reasoning with Rules / Search with Heuristics)

# Solving Real-World Problems



Real-World Problem

Formulation of Abstract Problem



Solve the Abstract Problem



Interpret the Solution



Implement the Solution



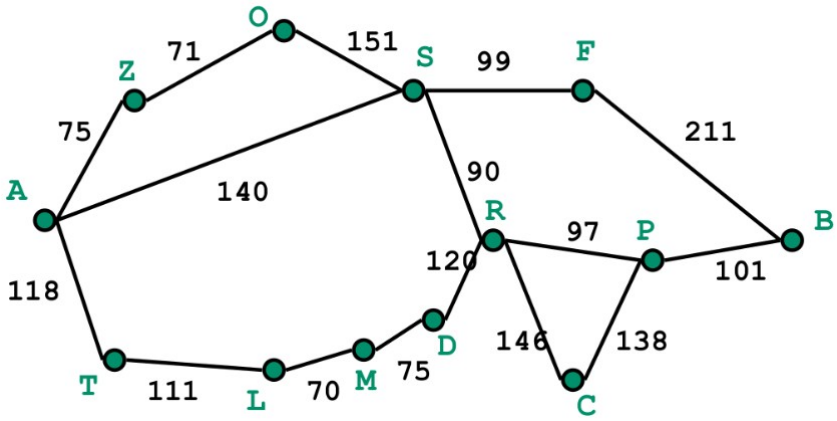
Credit: A. Lokketangen

# Solving Real-World Problems

## Find shortest route



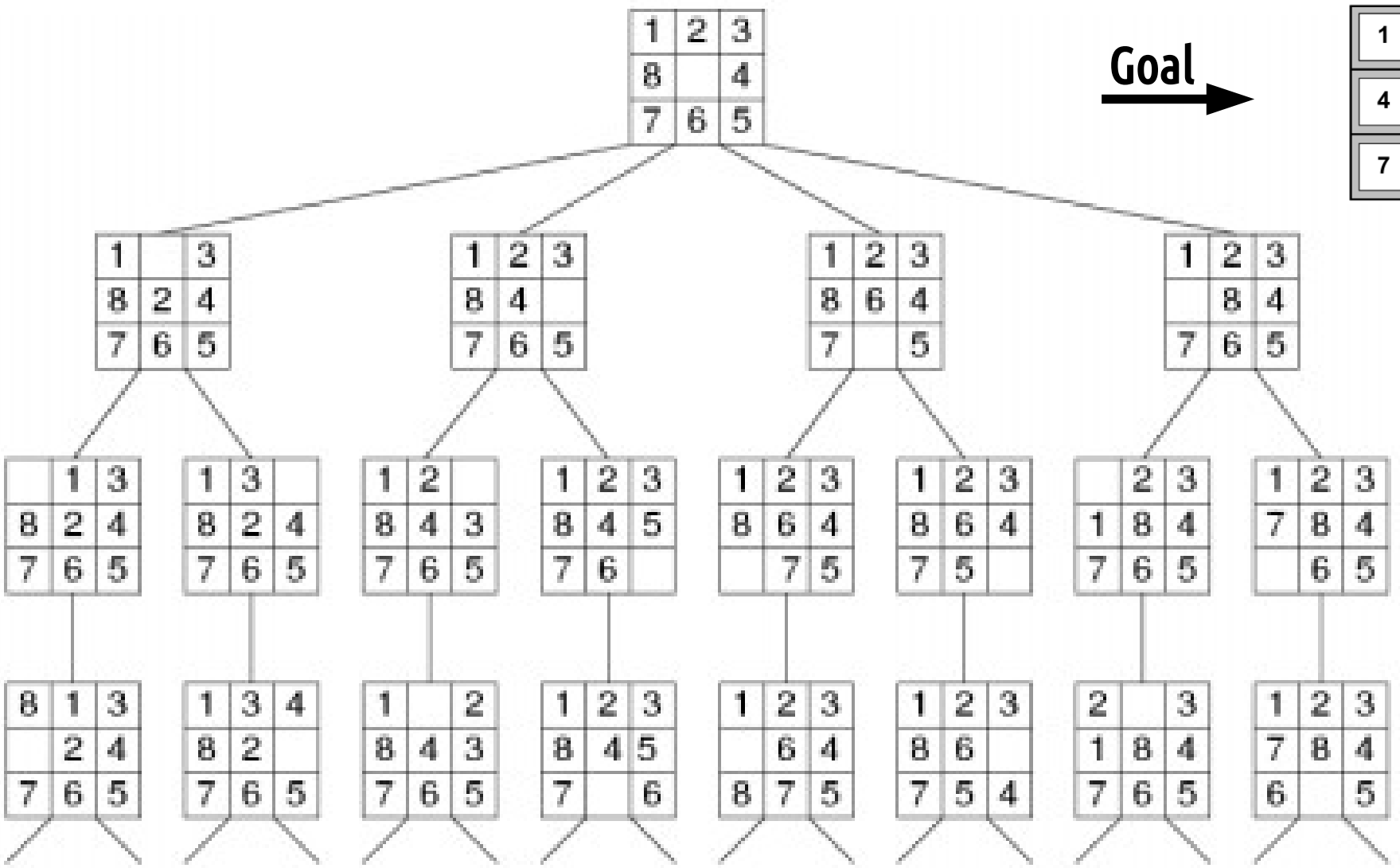

→ Formulate Abstract Problem




Solve the Abstract Problem

Credit: MIT, USA

# 8-Puzzle Problem

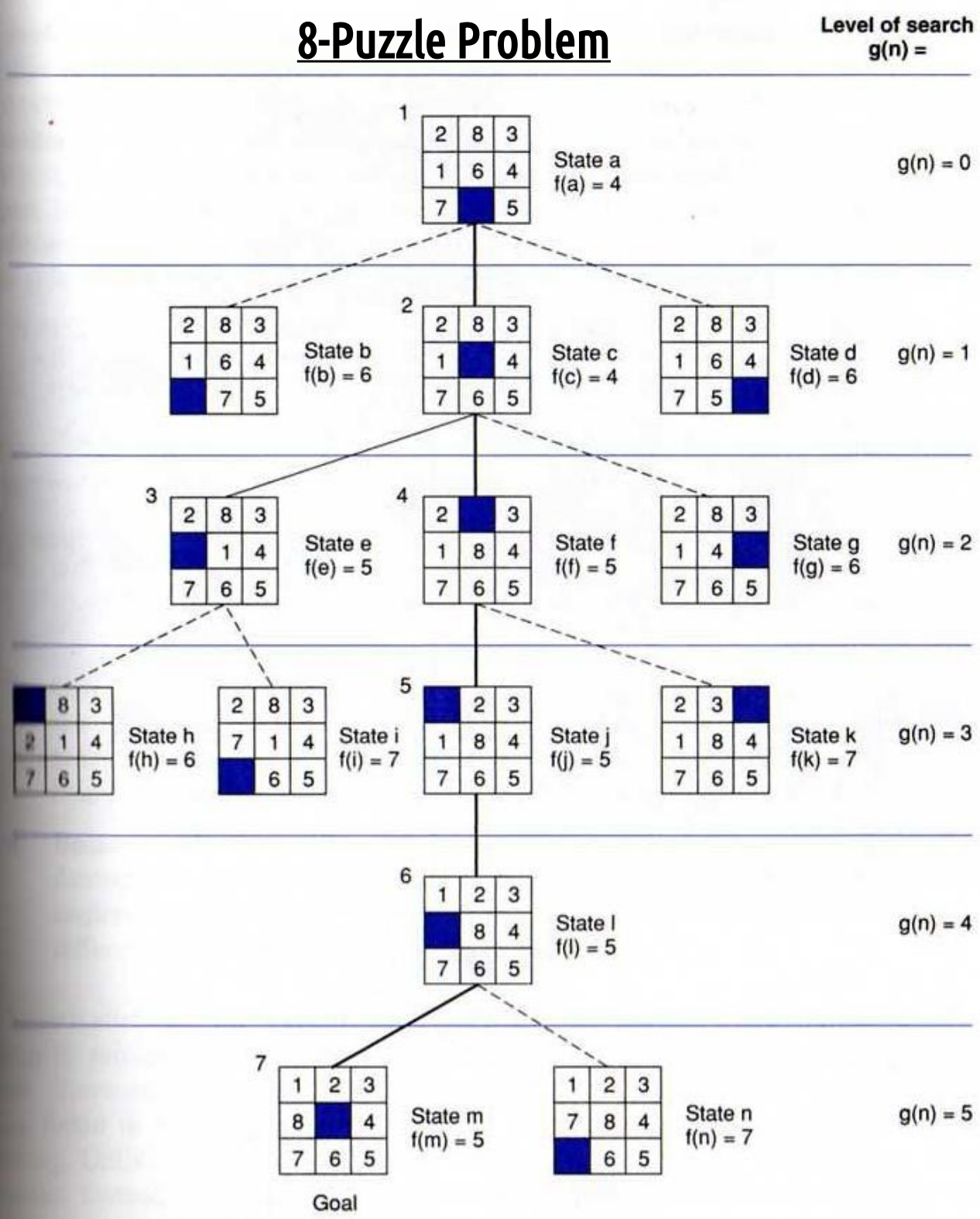



$d = 14$ IDS $=$ 3,473,941 nodes

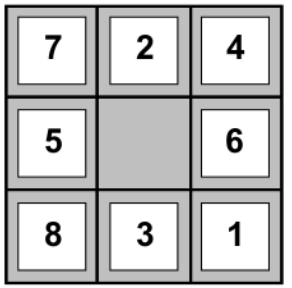
$d = 24$ IDS $\approx$ 54,000,000,000 nodes

Credit: MIT, USA

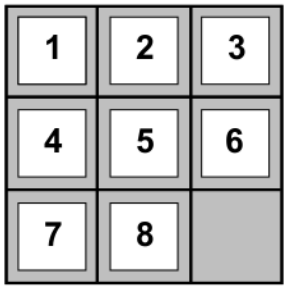
# 8-Puzzle Problem




# Heuristics



Start State



Goal State

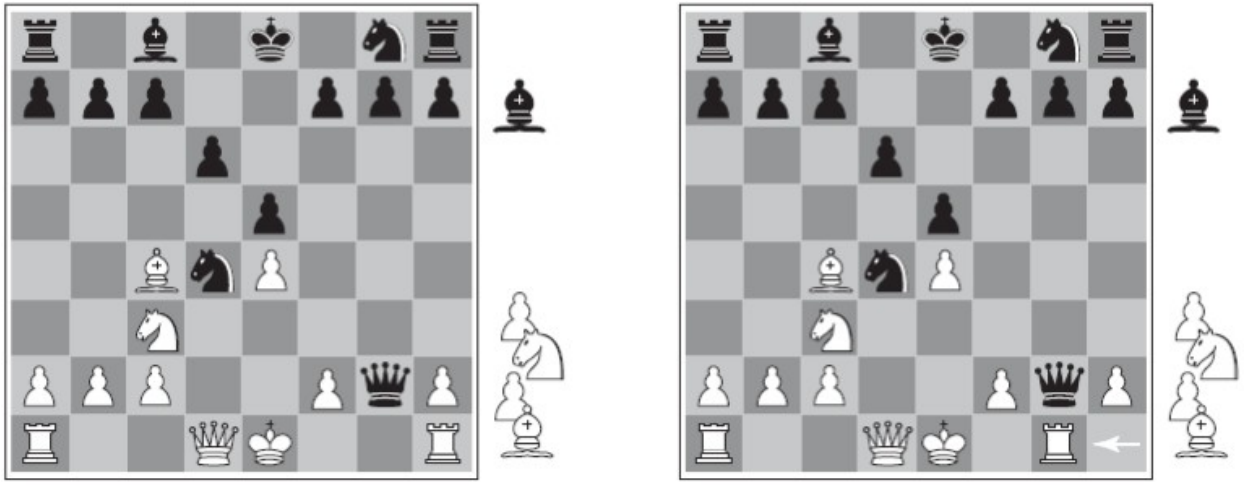
$h_1(n)$ = number of misplaced tiles
$h_2(n)$ = total Manhattan distance

$h_1(S)$ =?? 6
$h_2(S)$ =?? 4+0+3+3+1+0+2+1 = 14

$d = 14$ IDS = 3,473,941 nodes
$A^*(h_1)$ = 539 nodes
$A^*(h_2)$ = 113 nodes
$d = 24$ IDS $\approx$ 54,000,000,000 nodes
$A^*(h_1)$ = 39,135 nodes
$A^*(h_2)$ = 1,641 nodes

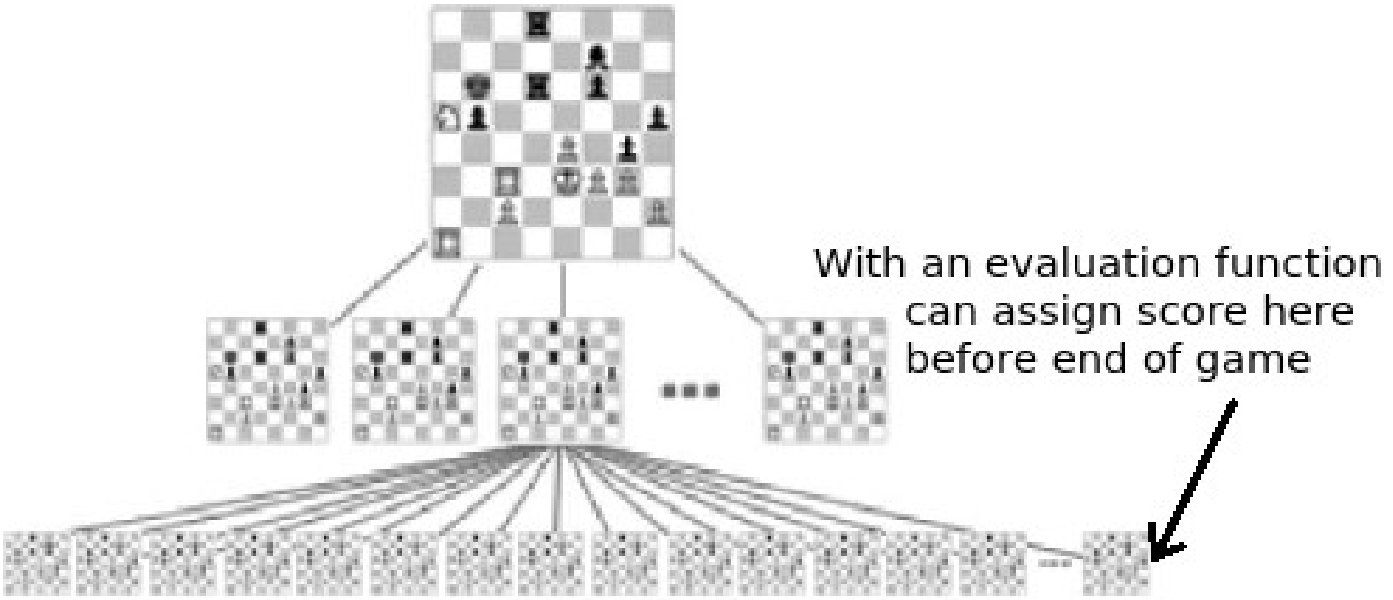
Credit: MIT, USA

# Chess Problem | Strategic Search



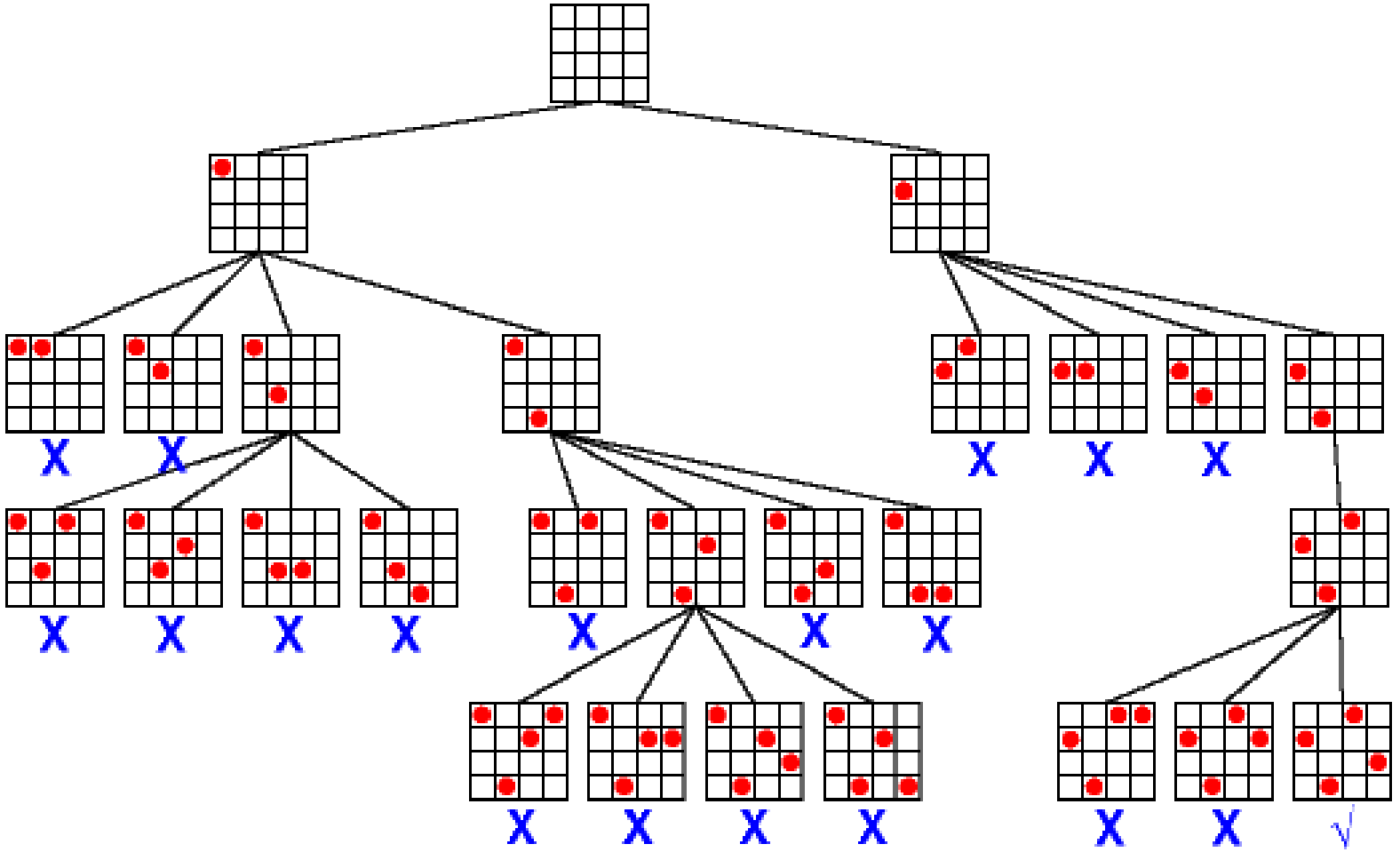
(a) White to move

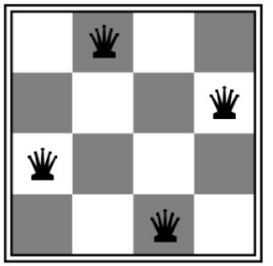
(b) White to move




Credit: MIT, USA

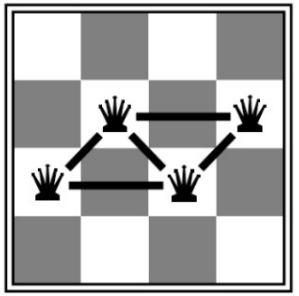
# N-Queen Problem | Local Search



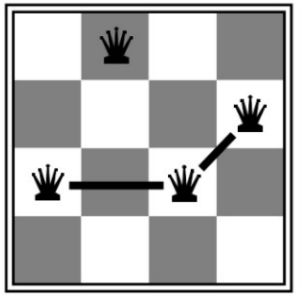


Goal

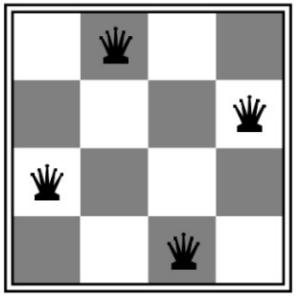
## Local Search:



h = 5



h = 2



h = 0

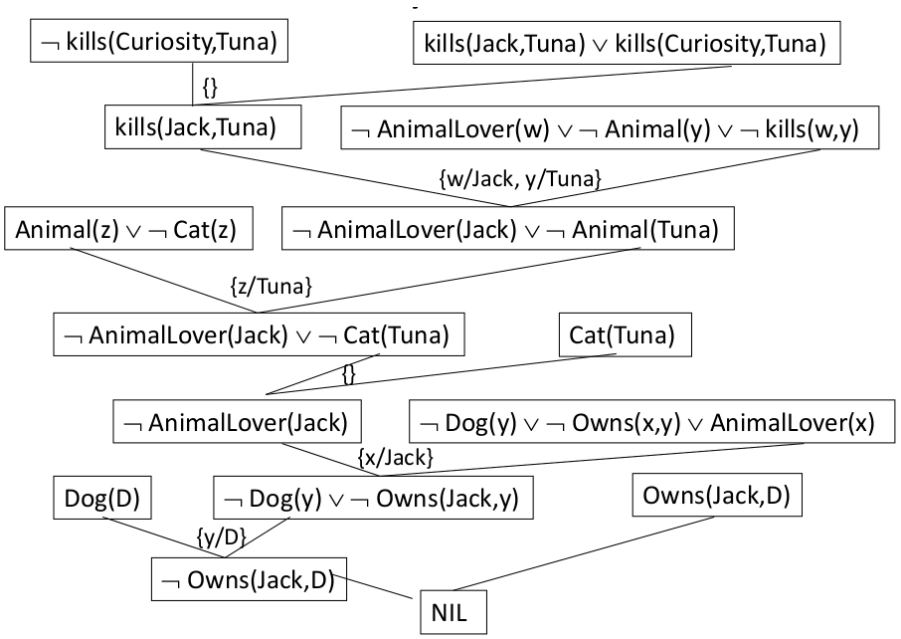
# Solving Linguistic Problems

- Jack owns a dog.
- Every dog owner is an animal lover.
- No animal lover kills an animal.
- Either Jack or Curiosity killed the cat, who is named Tuna.
- Did Curiosity kill Tuna?

## FOL Representation

A. $(\exists x)\ (Dog(x) \wedge Owns(Jack,x))$
B. $(\forall x)\ ((\exists y)\ (Dog(y) \wedge Owns(x, y))) \rightarrow AnimalLover(x))$
C. $(\forall x)\ (AnimalLover(x) \rightarrow ((\forall y)\ Animal(y) \rightarrow \neg Kills(x,y)))$
D. $Kills(Jack,Tuna) \vee Kills(Curiosity,Tuna)$
E. $Cat(Tuna)$
F. $(\forall x)\ (Cat(x) \rightarrow Animal(x))$
G. $Kills(Curiosity, Tuna)$ ◄—— Goal

## Solving by reasoning (resolution)

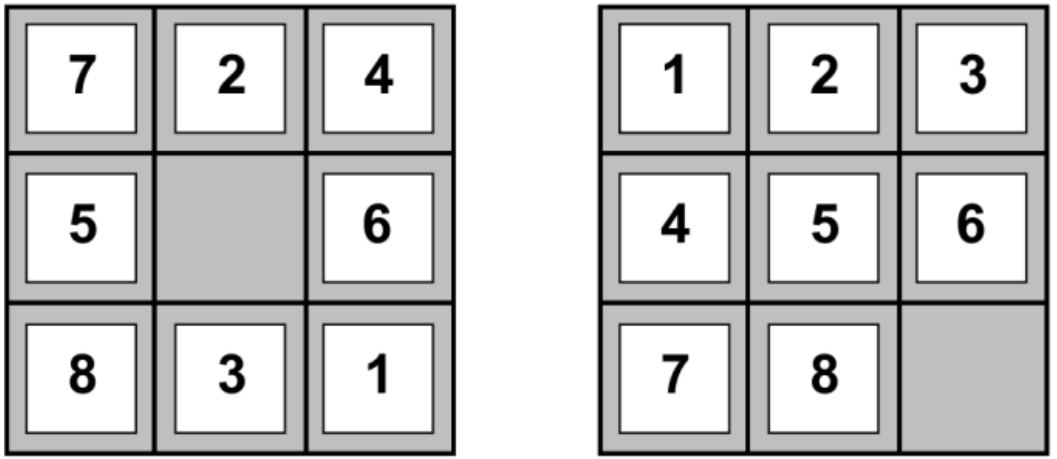


# Automated Planning (Modeling+Solving)

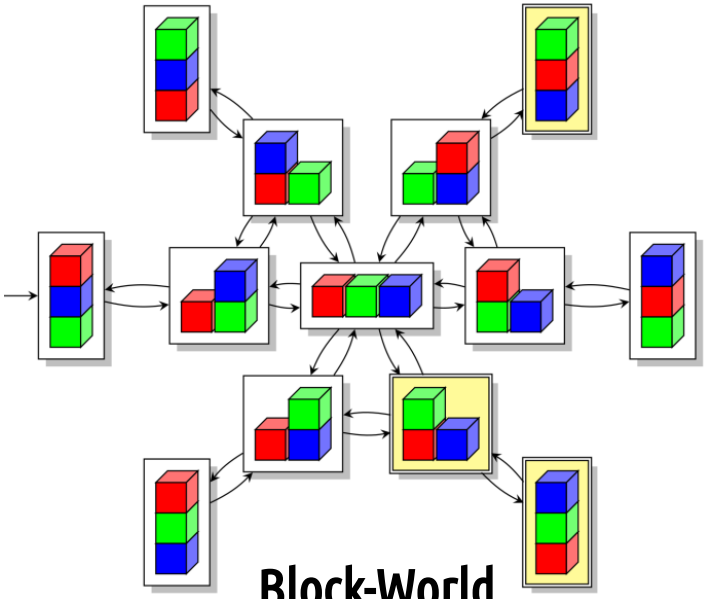
*Planning* is the task of finding a set of actions that will achieve a goal. A *planner* is a program that searches for a plan. It inputs a description of the world and the goals. The output is a plan. The simplest plan is a sequence of actions:

``do action1, do action2 ...''

More complex plans may include branching actions: "if (condition) do action1 else do action2"




Start State — 8-Puzzle Problem — Goal State



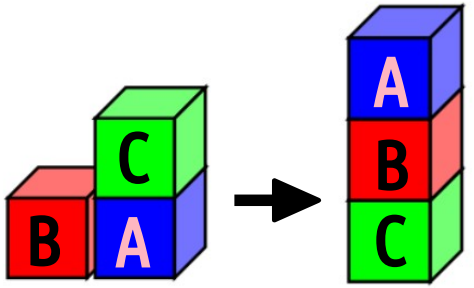
Block-World

# Automated Planning (Modeling: PDDL)

**Domain.pddl**

```
(define (domain blocks-world-domain)
  (:requirements :strips :equality :conditional-effects)
  (:constants Table)
  (:predicates (on ?x ?y)
               (clear ?x)
               (block ?b)
               )

  (:action move
          :parameters (?obj ?src ?dst)
          :precondition (and (on ?obj ?src) (clear ?obj) (clear ?dst)
                  (not (= ?dst ?src)) (not (= ?obj ?src))
                  (not (= ?obj ?dst)) (not (= ?obj Table)))
          :effect
          (and (on ?obj ?dst) (not (on ?obj ?src))
            (when (not (= ?src Table)) (clear ?src))
            (when (not (= ?dst Table)) (not (clear ?dst))))))
```




**init** **goal**

**Problem.pddl**

```
(define (problem sussman-anomaly)
    (:domain blocks-world-domain)
  (:objects A B C)
  (:init (block A) (block B) (block C) (block Table)
     (on C A) (on A Table) (on B Table) (clear C) (clear B) (clear Table))
  (:goal (and (on B C) (on A B))))
```

**Solution**

1) move C A Table
2) move B Table C
3) move A Table B

# Automated Planning (Modeling: PDDL)

**Domain.pddl**

```
(define (domain knights-tour)
  (:requirements :negative-preconditions)

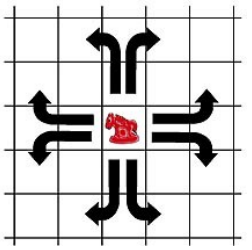
  (:predicates
    (at ?col ?row)
    (visited ?col ?row)
    (diff_by_one ?x ?y)
    (diff_by_two ?x ?y)
  )

  (:action move_2col_1row
   :parameters (?from_col ?from_row ?to_col ?to_row)
   :precondition (and (at ?from_col ?from_row)
              (diff_by_two ?from_col ?to_col) ; col +/- 2
              (diff_by_one ?from_row ?to_row) ; row +/- 1
              (not (visited ?to_col ?to_row)))
   :effect (and (not (at ?from_col ?from_row))
        (at ?to_col ?to_row)
        (visited ?to_col ?to_row))
   )

  (:action move_2row_1col
   :parameters (?from_col ?from_row ?to_col ?to_row)
   :precondition (and (at ?from_col ?from_row)
              (diff_by_two ?from_row ?to_row) ; row +/- 2
              (diff_by_one ?from_col ?to_col) ; col +/- 1
              (not (visited ?to_col ?to_row)))
   :effect (and (not (at ?from_col ?from_row))
        (at ?to_col ?to_row)
        (visited ?to_col ?to_row))
   )
)
```

| | | | | |
|---|---|---|---|---|
| 1 | 14 | 9 | 20 | 3 |
| 24 | 19 | 2 | 15 | 10 |
| 13 | 8 | 23 | 4 | 21 |
| 18 | 25 | 6 | 11 | 16 |
| 7 | 12 | 17 | 22 | 5 |

**Problem.pddl**



```
(define (problem knights-tour-problem-5x5)
  (:domain knights-tour)

  ; Define a set of "numbers" 1..8:
  (:objects n1 n2 n3 n4 n5)

  (:init
    ; Initial position of the K
    ; (upper left corner):
    (at n1 n5)
    (visited n1 n5)

    ; Here, we have to list all
    ; of the static predicates:
    (diff_by_one n1 n2)
    (diff_by_one n2 n1)
    (diff_by_one n2 n3)
    (diff_by_one n3 n2)
    (diff_by_one n3 n4)
    (diff_by_one n4 n3)
    (diff_by_one n4 n5)
    (diff_by_one n5 n4)

    (diff_by_two n1 n3)
    (diff_by_two n3 n1)
    (diff_by_two n2 n4)
    (diff_by_two n4 n2)
    (diff_by_two n3 n5)
    (diff_by_two n5 n3)
  )

  (:goal (and (visited n1 n1)
              (visited n1 n2)
              (visited n1 n3)
              (visited n1 n4)
              (visited n1 n5)
              (visited n2 n1)
              (visited n2 n2)
              (visited n2 n3)
              (visited n2 n4)
              (visited n2 n5)
              (visited n3 n1)
              (visited n3 n2)
              (visited n3 n3)
              (visited n3 n4)
              (visited n3 n5)
              (visited n4 n1)
              (visited n4 n2)
              (visited n4 n3)
              (visited n4 n4)
              (visited n4 n5)
              (visited n5 n1)
              (visited n5 n2)
              (visited n5 n3)
              (visited n5 n4)
              (visited n5 n5)))
)
```

# Constraint Satisfaction (Modeling: MiniZinc)

| | plot1 | plot2 | plot3 | plot4 | plot5 | plot6 | plot7 |
|---|---|---|---|---|---|---|---|
| barley | ✓ | ✓ | ✓ | – | – | – | – |
| corn | ✓ | – | – | ✓ | ✓ | – | – |
| millet | ✓ | – | – | – | – | ✓ | ✓ |
| oats | – | ✓ | – | ✓ | – | ✓ | – |
| rye | – | ✓ | – | – | ✓ | – | ✓ |
| spelt | – | – | ✓ | ✓ | – | – | ✓ |
| wheat | – | – | ✓ | – | ✓ | ✓ | – |

**Constraints** to be **satisfied**:

1. Equal growth load: Every plot grows 3 grains.
2. Equal sample size: Every grain is grown in 3 plots.
3. Balance: Every grain pair is grown in 1 common plot.

**Instance**: 7 plots, 7 grains, 3 grains/plot, 3 plots/grain, balance 1.

**AED.mzn**

```
int: nbrBlocks;
int: nbrVarieties;
set of int: Blocks = 1..nbrBlocks;
set of int: Varieties = 1..nbrVarieties;
int: blockSize;
int: sampleSize;
int: balance;
array[Varieties, Blocks] of var 0..1: BIBD;
solve satisfy;

constraint forall(b in Blocks)
(blockSize = sum(v in Varieties)(BIBD[v, b]));

constraint forall(v in Varieties)
(sampleSize = sum(b in Blocks)(BIBD[v, b]));

constraint forall(v, w in Varieties where v < w)
(balance = sum(b in Blocks)(BIBD[v, b] * BIBD[w, b]));

output[show(BIBD[v,w]) ++ if w == nbrBlocks then "\n"
  else " " endif | v in Varieties, w in Blocks]
```

**AED.dzn**

```
nbrBlocks = 7;
nbrVarieties = 7;
blockSize = 3;
sampleSize = 3;
balance = 1;
```

```
Finished in 60msec
Compiling aed-array.mzn
Running aed-array.mzn
1 0 1 1 0 0 0
0 0 0 1 1 1 0
1 0 0 0 0 1 1
0 0 1 0 1 0 1
0 1 0 1 0 0 1
0 1 1 0 0 1 0
1 1 0 0 1 0 0
----------
```

**solution**

# Discrete Optimisation (Modeling: MiniZinc)

**Four male celebrities are marooned on a desert island with five female celebrities. The 'happiness matrix' indicate how much happiness each couple would experience if the spend all their time together. The happiness earned by a couple is proportional to the fraction of time the spend together. For example, if Jon and Jolie spend half their time together, they earn happiness of (1/2)x(14) = 7. The optimal solution requires that each person spend all their time with one person of the opposite sex, so this result is often referred to as the Marriage Theorem.**

**The goal of the eight celebrities is to maximize the total happiness of the celebrities on the island. Determine the marriage partner for each celebrity.**

| Happiness | Jolie | Taylor | Kirsten | Lauren | Emma |
|---|---|---|---|---|---|
| Jon | 14 | 5 | 8 | 7 | 15 |
| Owen | 2 | 12 | 6 | 5 | 3 |
| Pachino | 7 | 8 | 3 | 9 | 7 |
| Tom | 2 | 4 | 6 | 10 | 1 |

```
total_cost: 44
0  0  0  0  1
0  1  0  0  0
1  0  0  0  0
0  0  0  1  0
----------
==========
Finished in 97msec
```

solution

## Maximize the total happiness of the celebrities

```
int: rows = 4;
int: cols = 5;
array[1..rows, 1..cols] of var 0..1: x;
array[1..rows, 1..cols] of int:  cost;
var int: total_cost =
  sum(i in 1..rows, j in 1..cols) (x[i,j]*cost[i,j]);

% maximize total_cost
solve maximize total_cost;

% exacly one assignment per row
constraint
 forall(i in 1..rows) (
        sum(j in 1..cols) (x[i,j]) = 1
 );

% zero or one assignments per column
constraint
  forall(j in 1..cols) (
        sum(i in 1..rows) (x[i,j]) <= 1
  );

% happiness table: cost
cost =
[|14,  5, 8,  7, 15,
 | 2, 12, 6,  5,  3,
 | 7,  8, 3,  9,  7,
 | 2,  4, 6, 10,  1
 |];
```